

Concurrent VHDL

Module Convenor: Dr. T. Nandha Kumar

Email: nandhakumaar.t@nottingham.edu.my

Department of Electrical & Electronic Engineering
The University of Nottingham Malaysia Campus



Introduction

- Concurrent VHDL Code = VHDL codes that are executable in parallel ie at the same clock cycle
- Topics to be covered in this class
 - ❑ Various concurrent statements
 - ❑ Behaviour of concurrent statements
 - ❑ Applications



VHDL Operators

GROUP	SYMBOL	FUNCTION
Arithmetic (binary)	+ - * / mod rem **	Addition Subtraction Multiplication Division Modulus Remainder Exponentiation
Arithmetic (unary)	+ - abs	Unary plus Unary minus Absolute value



VHDL Operators

GROUP	SYMBOL	FUNCTION
Relational	=	Equal
	/=	Not equal
	<	Less than
	>	Greater than
	<=	Less than or equal
	>=	Greater than or equal
Logical (binary)	AND	Logical and
	OR	Logical or
	NAND	Complement of and
	NOR	Complement of or
	XOR	Logical ex-or
Logical (unary)	NOT	Complement
Concatenation	&	Concatenation

Signal Assignment

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity full_adder_behav is
    Port ( a, b, cin : in std_logic;
          sum, cout : out std_logic);
end full_adder_behav;

architecture Behavioral of full_adder_behav is

begin
    sum <= a xor b xor cin;
    cout <= a and b and cin;

end Behavioral;
```

- The symbol '<=' is called Concurrent SIGNAL assignment.
- The values on the RHS are read, operated and passed to LHS with a delay
- The signals/ ports on the RHS should be 'in' or 'inout' or 'buffer' mode and never be in a 'out' mode

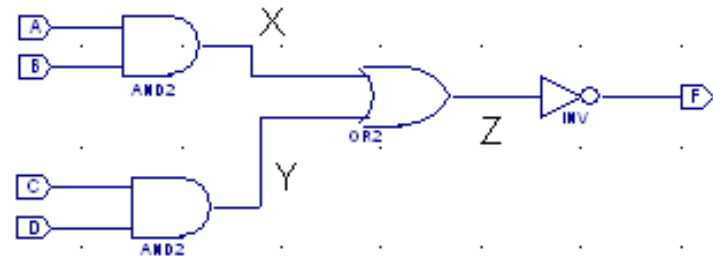
Syntax : <target_identifier> <= <selected_expression>;

Internal Signals

- Syntax for signal:
Architecture Identifier **of** EntityName **is**
--signal, variable declarations, etc
begin
--statements
end architecture Identifier;
- The region between '**is**' and '**begin**' in the architecture body is used to declare:
 - data types, constants, additional signals, components (as we have already seen)

Internal Signals (cont...)

```
library ieee;
use ieee.std_logic_1164.all;
--entity declaration
entity AOI is
port (A,B,C,D : in std_logic;
      F: out std_logic);
end entity AOI;
--architecture body
architecture demo of AOI is
signal X, Y, Z: std_logic;           --signal declaration including 'data type'
begin
    X<= A and B;                     --concurrent signal assignments
    Y<= C and D;                     --concurrent signal assignments
    Z<= X or Y;                       --concurrent signal assignments
    F<= not Z;                       --concurrent signal assignments
end architecture demo;
```



- Note: An internal signal is automatically of mode '**inout**' and hence can be read
- `Z <= X or Y;`
- and `F <= not Z;` --assign o/p to i/p
- Internal signals are very useful when the output is required to be read

Conditional signal statements

■ When – Else

```
library ieee;
use ieee.std_logic_1164.all;
--entity declaration
entity AND2 is
    port (A,B :in std_logic;
          Q:out std_logic);
end entity AND2;
--architecture body
architecture condnl of AND2 is
begin
    Q<=
        '1' after 6ns when A='1' and B='1' else
        '0' after 4ns;
end architecture condnl;
```


Conditional signal statements (cont...)

■ When – Else (cont...)

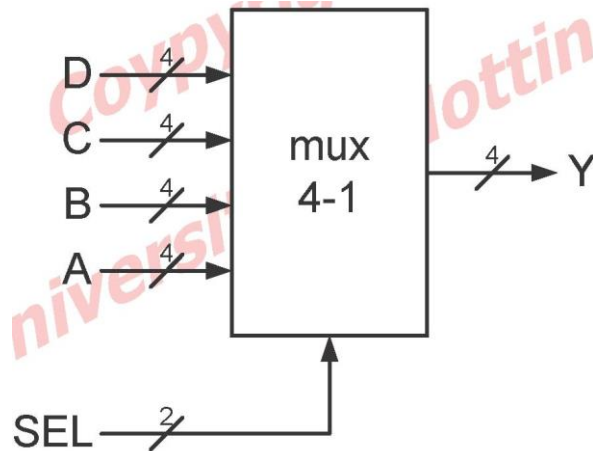
Syntax:

```
<target> <= <expression 1> WHEN <condition1> else  
.....  
.....  
<expression n-1> WHEN <condition n-1> else  
<expression n> ;
```

Note: each line doesn't end with ‘;’

Conditional signal statements (cont...)

■ When – Else (cont...)



Note: else implies a priority listing i.e first A, then B, then C, then finally D

■ Consider a more complete example:

```
--demonstration of a 4 bit wide 4-1 multiplexer 1
library ieee;
use ieee.std_logic_1164.all;
--entity declaration
entity mux is
    port(sel: in std_logic_vector (1 downto 0);
          A,B,C,D :in std_logic_vector (3 downto 0);
          Y:out std_logic_vector (3 downto 0));
end entity mux;
--architecture body
architecture condnl of mux is
begin
    Y <= A when sel="00" else
        B when sel="01" else
        C when sel="10" else
        D;
        --**
end architecture condnl;
```

Conditional signal statements (cont...)

- The last line (marked **) at first sight appears to be the final condition of **sel="11" then Y<=D;**
- Remember this is **std_logic** hence other values of sel exist: "1X" etc.
- Instead of this last line, VHDL compilers will accept:
 - ❑ **D when others;** --where sel can have any 'other' value.

Conditional signal statements (cont...)

- We can see priority more clearly in this next example of a priority encoder:

--demonstration of a priority encoder 1

library ieee;

use ieee.std_logic_1164.all;

--entity declaration

entity priority **is**

port (req1, req2, req3: **in** std_logic;

Y: **out** std_logic_vector (1 **downto** 0));

end entity priority;

--architecture body

architecture condn1 **of** priority **is**

begin

Y<="01" **when** req1= '1'**else**

"10" **when** req2= '1'**else**

"11" **when** req3= '1'**else**

"00";

end architecture condn1;

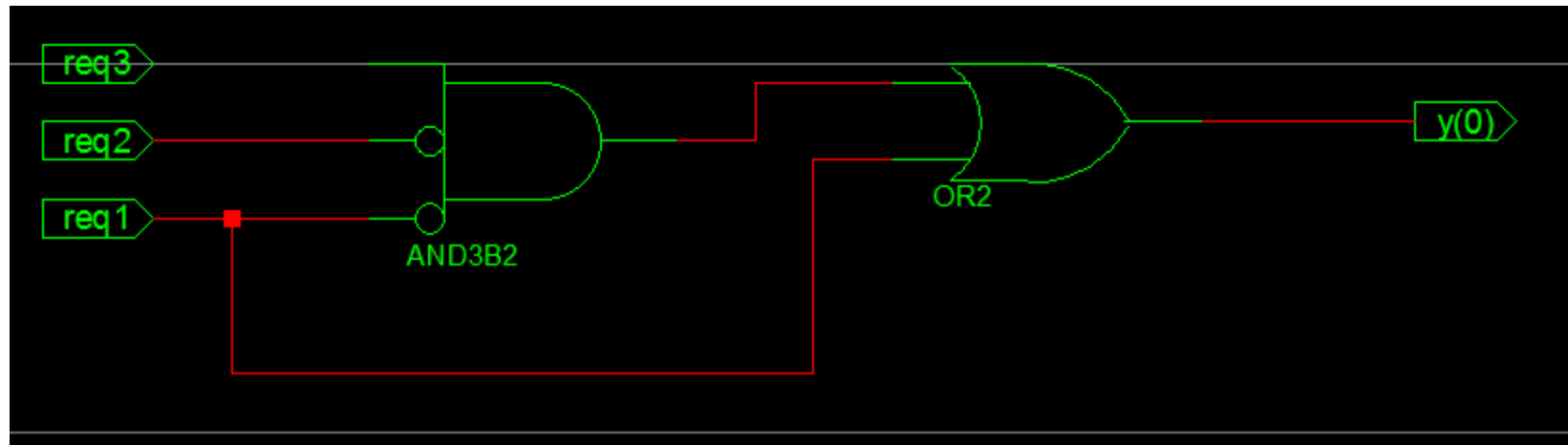
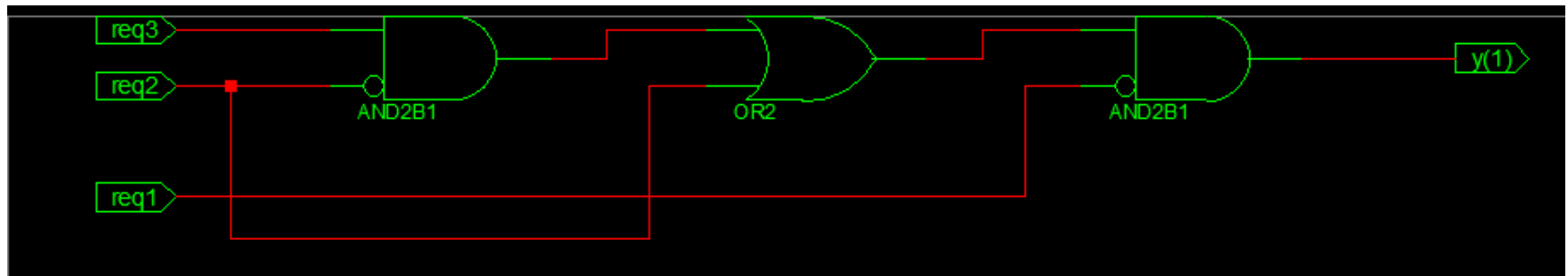
Note:

Here, we can see that '**req1**' has a higher priority than '**req2**' which in turn has a higher priority than '**req3**'

Hence **if** req3='1'**then** Y<="11" but only if neither req1 and req2 are not '1'

If none of the three inputs is 1 then Y<="00"

Conditional signal statements (cont...)



Selected signal Statement

■ With – Select

- Consider the same 4-1 Multiplexer example again but this time using the **selected signal assignment**:

--selected signal assignment of a 4 bit wide 4-1 multiplexer1

library ieee;

use ieee.std_logic_1164.all;

entity mux2 **is** --entity declaration

port (sel: **in** std_logic_vector (1 **downto** 0);

 A,B,C,D : **in** std_logic_vector (3 **downto** 0);

 Y: **out** std_logic_vector (3 **downto** 0));

end entity mux2;

architecture selected **of** mux2 **is** --architecture body

begin

with sel **select**

 Y<=A **when** "00", --when sel="00" then Y<=A

 B **when** "01", --etc

 C **when** "10",

 D **when others**; --**

end architecture selected;

Two Gate Delay

15

Selected signal Statement (cont...)

- A selected signal assignment is an assignment with no implied priority
- Very similar to the conditional signal assignment
- General form (inside arch body):
With expression **select**
 target <=expression1 **when** choice1,

 expressionN **when** choiceN;
- Since no '**else**' exists then all the expressions are evaluated and no priority exists

Selected signal Statement (cont...)

- We can use the keyword, '**unaffected**'
- 'the output is not to change'
- It can be used in a multiplexor:

With sel select

```
Y <=    A when "00",  
        B when "01",  
        C when "10",  
        unaffected when others;
```

- However, we should be aware that the synthesiser will see a latch or memory element called for here.
- Because the synthesised circuit must maintain the value of the output Y when the value of sel is '11'
- Implied memory
 - ❑ 5 gates used per memory element
 - ❑ Beware, if we are trying to save gates -more later on this.

Selected signal Statement (cont...)

- Another example of a selected signal assignment is shown below:
--demonstration of a 2-1 multiplexer1
library ieee;
use ieee.std_logic 1164.**all**;
--entity declaration
entity mux21 **is**
 port(x1, x2, sel: **in** std_logic;
 Y: **out** std_logic);
end entity mux21;
--architecture body
architectures elected **of** mux21 **is**
begin
 with sel **select**
 Y <= x1 **when** '0',
 x2 **when** **others**;
end architecture selected;

Hex – 7 Segment Decoder

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity HEX2LED is
5     port (
6         HEX:      in      STD_LOGIC_VECTOR (3 downto 0);
7         LED:      out     STD_LOGIC_VECTOR (6 downto 0)
8     );
9 end HEX2LED ;
10
11 architecture ARCH_NAME of HEX2LED is
12 begin
13     --HEX-to-seven-segment decoder
14     --  HEX: in  STD_LOGIC_VECTOR (3 downto 0);
15     --  LED: out STD_LOGIC_VECTOR (6 downto 0);
16     --
17     -- segment encoding
18     --      0
19     --      ---
20     --  5 |   | 1
21     --      ---  <- 6
22     --  4 |   | 2
23     --      ---
24     --      3
```

```
25     with HEX select
26         LED<=      "1111001" when "0001",    --1
27                   "0100100" when "0010",    --2
28                   "0110000" when "0011",    --3
29                   "0011001" when "0100",    --4
30                   "0010010" when "0101",    --5
31                   "0000010" when "0110",    --6
32                   "1111000" when "0111",    --7
33                   "0000000" when "1000",    --8
34                   "0010000" when "1001",    --9
35                   "0001000" when "1010",    --A
36                   "0000011" when "1011",    --b
37                   "1000110" when "1100",    --C
38                   "0100001" when "1101",    --d
39                   "0000110" when "1110",    --E
40                   "0001110" when "1111",    --F
41                   "1000000" when others;    --0
42
43 end ARCH_NAME ;
44
```

When – Else Vs With Select

- Conditional signal assignment is
 - Easier to use
 - Creates a larger amount of logic
 - Hence the chip is larger
- Select signal assignment uses less logic

& and Constant

■ & - means concatenation

- Eg vector d has the value “101”

vector b has the value “110”

Let $c \leftarrow d \& b$;

Then $c = \text{“101110”}$

■ Constant

- Doesn't change the value
- Declared in all parts
- Can be assign all data type

Constant $a_in: \text{std_logic} := \text{“110”}$;

Generic

- It must be declared in the entity before the port command
- It can have any data type

```
library ieee;  
use ieee.std_logic_1164.all;  
--entity declaration  
entity AND2 is  
    generic (delay :time := 10 ns)  
        port (A,B :in std_logic;  
            Q: out std_logic);  
end entity AND2;  
--architecture body  
architecture condn1 of AND2 is  
begin  
    Q<=  
        '1' after delay when A='1' and B='1' else  
        '0' after 4ns;  
end architecture condn1;
```

Generic map

- Used to design components which can be parameterized
- Generic map is used to change the value of generic component during instantiation

Entity *ex_gen* is

```
port ( d1, d2, d3, d4, d5 : in bit;  
       q1, q2: out bit);
```

End *ex_gen*;

Architecture *gen_behv* of *ex_gen* is

component *and_comp*

```
generic ( delay : time;  
         n : positive);
```

```
port ( a : in bit_vector ( n-1 downto 0);  
       c : out bit);
```

End component *and_comp*;

For *u1, u2* : *and_comp* use entity *work.and_comp*(*and_beh*);
begin

```
u1 : and_comp generic map (n=>2, delay=>9ns)port map(a(0)=>d1, a(1)=>d2, c=>q1);
```

```
u2 : and_comp generic map (n=>3, delay=>14ns)port map(a(0)=>d3, a(1)=>d4, a(2)=>d5, c=>q2);
```

End *gen_behv*;

Port Map

- It is used to link other components to an instanced component
- Syntax
port map (<port-association-list>);

Eg.

Entity *top_level* is

port (*a1*, *a2*: in *std_logic*;
 y1 : out *std_logic*);

End *top_level*;

Architecture *rtl_dff* of *top_level* is
component *dff*

port (*d1*, *d2* : in *std_logic*;
 q : out *std_logic*);

end component *dff*;

For *u1*: *dff* use entity *work.dff(rtl)*;

begin

u1 : *dff* port map (*a1*, *a2*, *y1*);

End *rtl_dff*;

Another method:

u1: *dff* port map (*d1*=>*a1*, *d2*=>*a2*, *q*=>*y1*);

Configuration

- It is used to bind an entity and an architecture
- Normally it is ignored by the synthesis tool

Syntax:

```
configuration configuration_name of entity_name is  
  for architecture_name  
    end for;  
end configuration_name;
```

- Two types – 1.Behavioural , 2. Structural

Configuration (Cont...)

- Behavioral
 - It is straight forward

```
configuration cfg_one of mux2 is
  for sequential
  end for;
end cfg_one;
```

```
configuration cfg_two of mux2 is
  for concurrent
  end for;
end cfg_two;
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2 is
  Port ( a_in : in std_logic;
         b_in : in std_logic;
         sel : in std_logic;
         mux_out : out std_logic);
end mux2;

architecture sequential of mux2 is
begin
  if sel = '0' then
    mux_out <= a_in;
  else
    mux_out <= b_in;
  end if;
end sequential;

architecture concurrent of mux2 is
begin
  with sel select
    mux_out <= a_in when '0',
              <= b_in when others;
end concurrent;
```

Configuration (Cont...)

■ Structural

```
entity mux2 is
    Port ( a : in std_logic;
          b : in std_logic;
          sel : in std_logic;
          y : out std_logic);
end mux2;

architecture Structure of mux2 is
    -- local components declaration
    component AND2 is
        port ( a,b : in std_logic; f: out std_logic);
    end component;

    component OR2 is
        port (a,b : in std_logic; f: out std_logic);
    end component OR2;

    component INV is
        port (a : in std_logic; f: out std_logic);
    end component;

    signal int1, int2, int3 : std_logic;
begin
    U1: INV    port map (sel, int1);
    U2: AND2   port map (sel,a,int2);
    U3: AND2   port map (int1,b,int3);
    U4: OR2    port map (int2,int3,y);

end Structure;
```

```
entity AND2 is
    port (A, B : in std_logic;
          F: out std_logic);
end AND2;
Architecture component_AND2 of AND2 is

    F <= A AND B;
end component_AND2;

entity OR2 is
    port (A,B : in std_logic;
          F : out std_logic);
end OR2;
Architecture component_OR2 of OR2 is

    F <= A OR B;
end component_OR2;

entity inv is
    port ( A : in std_logic;
          F : out std_logic);
end inv;
Architecture component_inv of inv is
    F <= NOT A;
end component_inv;

configuration structure_mux2 of mux2
    for STRUCTURE
        for U1: INV
            use entity WORK.INV(component_inv);
        end for;

        for U2, U3: AND2
            use entity WORK.AND2(component_AND2);
        end for;

        for U4: OR2
            use entity WORK.OR2(component_OR2);
        end for;
    end for;
end structure_mux2;
```



Questions

1. What does $\leq$ signify?
2. Which of the following type signals can be read?
in, out, inout
3. Define 'concurrent'?
4. Name the two different delay models used during simulation
5. Explain the difference between these different delay models.
6. Name the four different concurrent signal assignment statements.
7. How do we specify delay in a concurrent statement?
8. Which of these are legal concurrent signal assignments:
 $Y \Rightarrow a$; $Y \leq a$; $Y \leq "0"$; $Y \leq "1011"$;





True/False

1. A mode **out** can be read ?
2. A mode **in** can be assigned ?
3. The statement $V \leq '10X0'$; is that ok?
4. A conditional signal assignment has no priority?
5. A selected signal assignment must have mutually exclusive events in its when clause ?
6. When a signal changes then all concurrent statements are updated ?
7. A selected signal assignment uses less logic ?
8. The '**after**' statement is synthesisable?

Exercise 4

1. Correct the code and synthesis it.

```
Z => A when ASSIGN_A = "1"else;  
      B when ASSIGN_B = "1"else;  
      C;
```

2. Correct the code and synthesis it.

```
with CONTROL select  
      Z <=   A when '00';  
             B when '01';  
             C when '10';  
             D when '11';
```

Exercise 4

3. Differentiate between the MOD & REM
4. Differentiate between ' $\leq$ ' (assign) & ' $\leq$ ' (less than equal)
5. Given a 4-to-1 multiplexer shown in Figure Q5, with the entity provided as below:

```
entity multiplexer is
  port (
    SEL: in bit_vector (1 downto 0);
    A, B, C, D: in bit_vector (3 downto 0);
    Y: out bit_vector (3 downto 0));
end multiplexer;
```

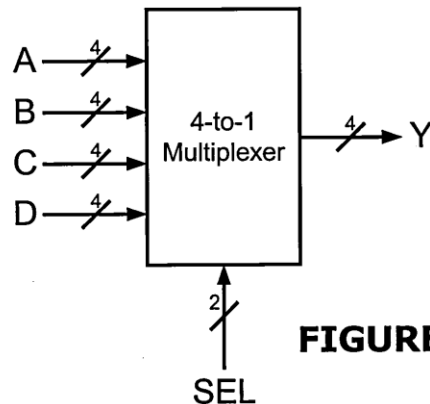


FIGURE Q5

Use selected signal assignment and case statements, respectively, to write **TWO** architectures that can work with the given entity. [10 Marks]



END of Lecture 4